23.3 Shell 工具与共享基础设施

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~140k tokens，输出 ~5k tokens（本节）

前两节我们分别解析了 exec 工具的命令执行引擎和 process 工具的进程管理机制。本节聚焦于支撑它们运行的**共享基础设施**：Shell 检测与配置、Docker 沙箱环境构建、工作目录解析、二进制输出清理、以及进程树管理。这些看似简单的工具函数，实则隐藏着大量的跨平台兼容性考量。

23.3.1 Shell 检测与配置

平台感知的 Shell 选择

`getShellConfig()` 是整个 Bash 工具链的入口——它决定了命令通过哪个 Shell 来执行：

```typescript
// src/agents/shell-utils.ts — getShellConfig
export function getShellConfig(): { shell: string; args: string[] } {
  if (process.platform === "win32") {
    return {
      shell: resolvePowerShellPath(),
      args: ["-NoProfile", "-NonInteractive", "-Command"],
    };
  }

  const envShell = process.env.SHELL?.trim();
  const shellName = envShell ? path.basename(envShell) : "";

  // Fish shell 会拒绝 bashism 语法，降级到 bash
  if (shellName === "fish") {
    const bash = resolveShellFromPath("bash");
    if (bash) return { shell: bash, args: ["-c"] };
    const sh = resolveShellFromPath("sh");
    if (sh) return { shell: sh, args: ["-c"] };
  }

  const shell = envShell?.length > 0 ? envShell : "sh";
  return { shell, args: ["-c"] };
}
```

三个关键设计决策：

1. Windows 使用 PowerShell 而非 cmd.exe

```typescript
function resolvePowerShellPath(): string {
  const systemRoot = process.env.SystemRoot || process.env.WINDIR;
  if (systemRoot) {
    const candidate = path.join(
      systemRoot, "System32", "WindowsPowerShell", "v1.0", "powershell.exe"
    );
    if (fs.existsSync(candidate)) return candidate;
  }
  return "powershell.exe";
}
```

> **衍生解释**：Windows 的 `cmd.exe` 有一个严重缺陷：许多系统工具（如 `ipconfig` 、 `systeminfo` ）通过 Win32 API `WriteConsole()` 直接向控制台输出，而不是通过标

> 准的 stdout 管道。当 Node.js 使用管道模式 spawn cmd.exe 时，这些工具的输出会"消失"。PowerShell 能正确捕获并重定向这些输出，因此 OpenClaw 在 Windows 上选择 PowerShell。

2. Fish Shell 自动降级

Fish shell 的语法与 POSIX shell/bash 差异极大（例如不支持 `$()` 子命令替换、 `&&` 等），大部分工具生成的命令都假设 bash 语法。OpenClaw 检测到 Fish 时，尝试依次查找 `bash` → `sh` 来替代。

3. Shell 路径解析

```
function resolveShellFromPath(name: string): string | undefined {
  const entries = (process.env.PATH ?? "").split(path.delimiter);
  for (const entry of entries) {
    const candidate = path.join(entry, name);
    try {
      fs.accessSync(candidate, fs.constants.X_OK);
      return candidate;
    } catch { /* ignore */ }
  }
  return undefined;
}
```

手动遍历 PATH，用 `accessSync(X_OK)` 检查文件是否可执行。这比依赖 `which` 命令更可靠——`which` 本身就需要一个 Shell 来执行，形成循环依赖。

23.3.2 Docker 沙箱环境构建

buildDockerExecArgs — 构建 Docker exec 参数

```
// src/agents/bash-tools.shared.ts — buildDockerExecArgs
export function buildDockerExecArgs(params) {
  const args = ["exec", "-i"];           // -i: 保持 stdin 开放
  if (params.tty) args.push("-t");       // -t: 分配 TTY
  if (params.workdir) args.push("-w", params.workdir);  // 工作目录

  // 注入环境变量
  for (const [key, value] of Object.entries(params.env)) {
    args.push("-e", `${key}=${value}`);
  }

  // PATH 特殊处理
  if (hasCustomPath) {
    args.push("-e", `OPENCLAW_PREPEND_PATH=${params.env.PATH}`);
  }

  // 使用 login shell 并在 profile 加载后前置 PATH
  const pathExport = hasCustomPath
    ? 'export PATH="${OPENCLAW_PREPEND_PATH}:$PATH"; unset OPENCLAW_PREPEND_PATH; '
    : "";
  args.push(params.containerName, "sh", "-lc", `${pathExport}${params.command}`);
  return args;
}
```

这里有一个精妙的 PATH 处理技巧：

```
问题：Docker 容器内使用 login shell (-l) 时，/etc/profile 会重置 PATH
      导致通过 -e PATH=xxx 设置的自定义路径被覆盖

解决：
  1. 将自定义 PATH 存入临时变量 OPENCLAW_PREPEND_PATH
  2. 在命令前添加：export PATH="${OPENCLAW_PREPEND_PATH}:$PATH"
  3. 这样自定义路径在 profile 加载后才注入，不会被覆盖
  4. 最后 unset OPENCLAW_PREPEND_PATH 避免变量泄露
```

buildSandboxEnv — 构建沙箱环境变量

```
// src/agents/bash-tools.shared.ts — 沙箱环境变量
export function buildSandboxEnv(params) {
  const env: Record<string, string> = {
    PATH: params.defaultPath,        // 系统默认 PATH
    HOME: params.containerWorkdir,   // HOME 设为工作目录
  };
  // 先合并沙箱配置的 env，再合并用户指定的 env
  for (const [key, value] of Object.entries(params.sandboxEnv ?? {})) {
    env[key] = value;
  }
  for (const [key, value] of Object.entries(params.paramsEnv ?? {})) {
    env[key] = value;
  }
  return env;
}
```

合并顺序很重要：用户指定的环境变量（`paramsEnv`）优先级最高，可以覆盖沙箱默认值。沙箱模式下没有环境变量安全检查——因为容器提供了足够的隔离。

23.3.3 工作目录解析

宿主机模式

```
// src/agents/bash-tools.shared.ts — resolveWorkdir
export function resolveWorkdir(workdir: string, warnings: string[]) {
  const current = safeCwd();
  const fallback = current ?? homedir();
  try {
    const stats = statSync(workdir);
    if (stats.isDirectory()) return workdir;
  } catch { /* ignore */ }

  warnings.push(`Warning: workdir "${workdir}" is unavailable; using "${fallback}".`);
  return fallback;
}

function safeCwd() {
  try {
    const cwd = process.cwd();
    return existsSync(cwd) ? cwd : null;
  } catch { return null; }
}
```

`safeCwd()` 处理了一种边缘情况：如果当前工作目录被删除（例如另一个进程删除了目录），`process.cwd()` 会抛出异常。这在长时间运行的服务器中并非罕见。

沙箱模式

```
// src/agents/bash-tools.shared.ts — resolveSandboxWorkdir
export async function resolveSandboxWorkdir(params) {
  try {
    // 验证路径在沙箱允许范围内
    const resolved = await assertSandboxPath({
      filePath: params.workdir,
      cwd: process.cwd(),
      root: params.sandbox.workspaceDir,
    });
    // 转换为容器内路径
    const relative = resolved.relative
      ? resolved.relative.split(path.sep).join(path.posix.sep)
      : "";
    const containerWorkdir = relative
      ? path.posix.join(params.sandbox.containerWorkdir, relative)
      : params.sandbox.containerWorkdir;
    return { hostWorkdir: resolved.resolved, containerWorkdir };
  } catch {
    return {
      hostWorkdir: params.sandbox.workspaceDir,
      containerWorkdir: params.sandbox.containerWorkdir,
    };
  }
}
```

沙箱模式需要做**路径映射**：宿主机路径 → 容器内路径。例如：

```
宿主机路径:  /home/user/projects/myapp
沙箱工作区:  /home/user/projects
容器工作目录: /workspace

映射结果:
  hostWorkdir:      /home/user/projects/myapp
  containerWorkdir: /workspace/myapp
```

`path.sep` 到 `path.posix.sep` 的转换也不能忽略——如果宿主机是 Windows（反斜杠分隔符），容器内的路径仍然需要使用正斜杠。

23.3.4 二进制输出清理

```
// src/agents/shell-utils.ts — sanitizeBinaryOutput
export function sanitizeBinaryOutput(text: string): string {
  // 1. 移除 Unicode 格式字符和代理对
  const scrubbed = text.replace(/[\p{Format}\p{Surrogate}]/gu, "");

  // 2. 逐字符过滤
  const chunks: string[] = [];
  for (const char of scrubbed) {
    const code = char.codePointAt(0);
    // 保留 Tab(\x09)、换行(\x0a)、回车(\x0d)
    if (code === 0x09 || code === 0x0a || code === 0x0d) {
      chunks.push(char);
      continue;
    }
    // 过滤 ASCII 控制字符 (0x00-0x1f)
    if (code < 0x20) continue;
    chunks.push(char);
  }
  return chunks.join("");
}
```

当命令输出包含二进制数据（如 `cat` 一个图片文件）或乱码终端控制序列时， `sanitizeBinaryOutput` 确保传递给 Agent 的文本是合法的可显示字符串。它保留了制表符、换行、回车等必要的空白字符，但过滤掉了其他所有控制字符。

> **衍生解释**：Unicode 的 `\p{Format}` 类别包含零宽连接符（ZWJ）、方向控制字符（LTR/RTL 标记）等不可见字符； `\p{Surrogate}` 是 UTF-16 编码中的代理对，在 JavaScript 字符串中表示不完整的代理对（通常是编码错误）。这些字符如果传递给 LLM API，可能导致 JSON 序列化错误或产生不可预期的行为。

23.3.5 进程树管理

```
// src/agents/shell-utils.ts — killProcessTree
export function killProcessTree(pid: number): void {
  if (process.platform === "win32") {
    spawn("taskkill", ["/F", "/T", "/PID", String(pid)], {
      stdio: "ignore",
      detached: true,
    });
    return;
  }

  try {
    process.kill(-pid, "SIGKILL");  // 负 PID = 杀死整个进程组
  } catch {
    try {
      process.kill(pid, "SIGKILL"); // 回退: 只杀主进程
    } catch { /* 进程已死 */ }
  }
}
```

> **衍生解释**：在 Unix 系统中，使用 `detached: true` 创建的子进程会成为新进程组（Process Group）的组长。 `process.kill(-pid, signal)` 中的负 PID 表示向整个进程组发送信号，这样可以杀死进程及其所有子进程。例如执行 `npm run build` 时，npm 会 spawn node，node 可能 spawn webpack，webpack 再 spawn 多个 worker——只杀 npm 进程不够，需要杀掉整个进程树。Windows 上使用 `taskkill /T` （/T = tree）实现同样的效果。

`killSession()` 是对 `killProcessTree()` 的封装：

```
// src/agents/bash-tools.shared.ts
export function killSession(session) {
  const pid = session.pid ?? session.child?.pid;
  if (pid) killProcessTree(pid);
}
```

23.3.6 共享工具函数

Bash 工具链还包含一组实用的共享函数：

字符串截断

```
// 中间截断 — 保留首尾，中间用 ... 替代
export function truncateMiddle(str: string, max: number) {
  if (str.length <= max) return str;
  const half = Math.floor((max - 3) / 2);
  return `${sliceUtf16Safe(str, 0, half)}...${sliceUtf16Safe(str, -half)}`;
}
// truncateMiddle("very-long-command-name", 15) → "very-l...d-name"
```

使用 `sliceUtf16Safe()` 而非普通 `String.slice()` ——确保不会在 UTF-16 代理对的中间截断，导致产生无效的 Unicode 字符。

日志分页

```
// src/agents/bash-tools.shared.ts — 日志切片
export function sliceLogLines(text: string, offset?: number, limit?: number) {
  const lines = text.replace(/\r\n/g, "\n").split("\n");
  // 移除末尾空行
  if (lines[lines.length - 1] === "") lines.pop();

  let start = offset ?? 0;
  // 如果只指定 limit 不指定 offset，取最后 N 行（类似 tail -n）
  if (limit !== undefined && offset === undefined) {
    start = Math.max(lines.length - limit, 0);
  }

  return {
    slice: lines.slice(start, end).join("\n"),
    totalLines: lines.length,
    totalChars: text.length,
  };
}
```

当只传入 `limit` 时，默认行为类似 `tail -n` ——返回最后 N 行。这符合 Agent 的典型使用场景："看看最后几行输出"。

命令名称提取

```
export function deriveSessionName(command: string): string | undefined {
  const tokens = tokenizeCommand(command);
  const verb = tokens[0];
  let target = tokens.slice(1).find(t => !t.startsWith("-"));
  return target ? `${verb} ${truncateMiddle(stripQuotes(target), 48)}` : verb;
}
```

持续时间格式化

```
export function formatDuration(ms: number) {
  if (ms < 1000) return `${ms}ms`;
  const seconds = Math.floor(ms / 1000);
  if (seconds < 60) return `${seconds}s`;
  const minutes = Math.floor(seconds / 60);
  const rem = seconds % 60;
  return `${minutes}m${rem.toString().padStart(2, "0")}s`;
}
// 500 → "500ms"
// 3000 → "3s"
// 125000 → "2m05s"
```

23.3.7 桶文件与模块导出

最后， `bash-tools.ts` 作为桶文件（barrel file）将所有导出聚合：

```
// src/agents/bash-tools.ts — 完整文件
export type { BashSandboxConfig, ExecElevatedDefaults,
              ExecToolDefaults, ExecToolDetails } from "./bash-tools.exec.js";
export { createExecTool, execTool } from "./bash-tools.exec.js";
export type { ProcessToolDefaults } from "./bash-tools.process.js";
export { createProcessTool, processTool } from "./bash-tools.process.js";
```

> **衍生解释**：桶文件（Barrel File）是 TypeScript/JavaScript 项目中常见的模块组织模式。它本身不包含业务逻辑，仅从子模块重新导出符号，使消费者可以从一个入口导入所需内

> 容： `import { execTool, processTool } from "./bash-tools.js"` ，而不必知道内部的文件拆分。

外部代码只需 `import { execTool, processTool } from "./bash-tools.js"` 即可获取预配置的工具实例，或使用 `createExecTool(customDefaults)` 创建定制实例。

本节小结

1. **Shell 检测是平台感知的**：Windows 用 PowerShell（避免 WriteConsole 捕获问题）；Fish 降级到 bash（避免 bashism 不兼容）；默认使用 `$SHELL` 环境变量
2. **Docker PATH 注入使用临时变量技巧**——先存入 `OPENCLAW_PREPEND_PATH` ，login shell profile 加载后再 export，避免被 `/etc/profile` 覆盖
3. **工作目录解析处理多种边缘情况**：目录不存在、当前目录被删除、沙箱路径映射（宿主 → 容器）、Windows/Unix 路径分隔符转换
4. **二进制输出清理**移除 Unicode 格式字符、代理对和 ASCII 控制字符，确保 LLM 接收到的文本是合法的
5. **进程树杀死**利用 Unix 进程组（负 PID）和 Windows taskkill /T 来确保子进程链被完整终止
6. **共享工具函数**处理了 UTF-16 安全截断、tail-like 日志分页、命令名称提取和持续时间格式化
7. **桶文件模式**为外部消费者提供了简洁统一的导入入口